

GhostNet-AI: Strategy & Architecture

SIH26057 — AI-Powered Underwater Marine Debris and Anomaly Detection using Side-Scan Sonar

Yes. Now that I have your actual SIH26057 PPT and have checked the current public landscape around this problem, I would **change the strategy from “more features” to “a few defensible innovations that other YOLO + dashboard teams are unlikely to demonstrate.”**

One important reality: public 2026 examples already show **YOLO-based side-scan-sonar marine-debris detection, ONNX/edge deployment, and even synthetic ghost-net training for SIH26057.** ([Hugging Face](https://huggingface.co/rehan9599/drishti-detector?utm_source=chatgpt.com "rehan9599/drishti-detector · Hugging Face")) Also, shadow-aware sonar detection itself is already an active research direction. ([ScienceDirect](https://www.sciencedirect.com/science/article/pii/S0029801825014179?utm_source=chatgpt.com "Side-scan sonar mine-like target detection considering acoustic illumination and shadow characteristics - ScienceDirect"))

So simply saying:

YOLO + FastAPI + GIS + confidence

will probably not make your project look sufficiently different.

Your current PPT already has the right foundation—XTF processing, AI detection, uncertainty, geolocation, GIS, human review and reporting. The next step is to make **GhostNet-AI an “evidence and decision system,” not just an object detector.**

The core concept I recommend

GhostNet-AI = DETECT → VERIFY → LOCALIZE → PRIORITIZE

Instead of:

“AI detects a ghost net.”

your system should answer:

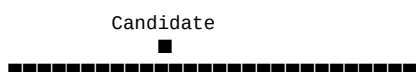
*****“What is this target, why do we believe it is artificial, how certain are we, where exactly is it, how much location uncertainty exists, and should an operator investigate it?”*****

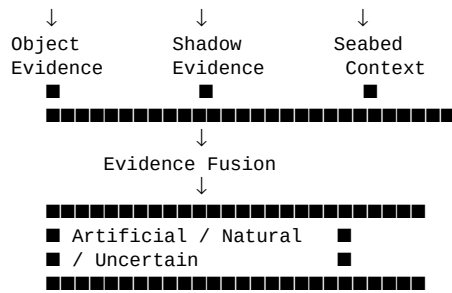
That is a much stronger product story.

1. Acoustic Evidence Fusion Engine — my strongest suggestion

Do not let YOLO make the final decision alone.

For every candidate, calculate several independent pieces of evidence:





Example

A detection might say:

YOLO confidence	88
Shadow consistency	High
Shape consistency	Medium
Seabed context	High
Sensor quality	Good

Final evidence state:
ARTIFICIAL – HIGH CONFIDENCE

Or:

YOLO confidence	91
Shadow consistency	Low
Shape consistency	Low
Seabed context	Natural ridge

Final evidence state:
UNCERTAIN → HUMAN REVIEW

That is much more interesting than simply displaying a bounding box.

The research literature already shows that acoustic shadow information can help reduce false alarms in SSS detection, so your differentiation should be **fusion + operational decisioning**, not the claim that “we invented shadow awareness.” ([ScienceDirect](https://www.sciencedirect.com/science/article/pii/S0029801825014179?utm_source=chatgpt.com "Side-scan sonar mine-like target detection considering acoustic illumination and shadow characteristics - ScienceDirect"))

2. Make the AI capable of saying “I DON'T KNOW”

This could become one of your signature ideas.

Most demos try to make AI output something every time.

Your system should have:

CONFIRM

High evidence

REVIEW

Conflicting evidence

ABSTAIN

Insufficient evidence

Example:

```

■ TARGET #024 ■
■
■ Object: Possible Ghost Net ■
■ Confidence: 82% ■
■ Evidence: Conflicting ■
■
■ ■ AI recommends review ■
■
■ [ ACCEPT ] [ REJECT ] ■

```

Your current PPT already mentions calibrated confidence, uncertainty and graceful degradation.

Take this much further in the actual system.

This gives you a very good line for the PPT:

****“GhostNet-AI does not force a prediction when the evidence is insufficient.”****

That is a strong engineering philosophy.

...

3. Track-level detection instead of frame-level detection

This is one of the best practical “jugaad” ideas.

A ghost net or wreck may appear in **many consecutive** sonar frames.

Instead of counting:

```
Frame 101 → target
Frame 102 → target
Frame 103 → target
Frame 104 → target
```

as four objects, track them together:

```
FRAME 101  ■■
FRAME 102  ■■
FRAME 103  ■■■■&gt; SAME TARGET
FRAME 104  ■■
```

Then create:

Target #024

- Seen in 17 frames
- Spatially consistent

- Average confidence: X
- Best frame: Y
- Estimated location
- Location uncertainty
- Final consolidated record

This makes your system feel like **survey intelligence**, not image classification.

— — —

4. One target = one “Evidence Dossier”

This could be your signature UI.

When an operator clicks any detection:

TARGET DOSSIER #024

[illegible]

TYPE
Possible Ghost Net

AI confidence
82%

Evidence	
Object shape	✓
Shadow	✓
Seabed context	?
Track consistency	✓
Sensor quality	✓

LOCATION
21.4721° N
72.1842° E

Location uncertainty
± 8.4 m

STATUS
NEEDS HUMAN REVIEW

[ACCEPT] [REJECT] [MARK UNKNOWN]

Then:

Export Evidence Report

This gives the project a **forensic / operational** feel.

— — —

5. Add “Why did the AI flag this?”

Not generic explainable AI.

Make it sonar-specific.

For example:

Why flagged?

- ✓ Strong acoustic return
- ✓ Consistent target geometry
- ✓ Shadow attached to target
- ✓ Persistent across 11 frames
- ✓ Not matching nearby natural seabed pattern

And for a false positive:

Why uncertain?

- ✓ Object-like return
- ✗ Weak/absent shadow
- ✗ Similar to natural ridge
- ✗ Poor sonar quality

That turns your AI from a black box into an **investigation assistant**.

6. Sonar Quality Gate — very good differentiator

Before running detection, let the system assess:

Is this sonar segment trustworthy?

SONAR QUALITY

Noise	■■■■■
Contrast	■■■■■
Shadow clarity	■■■■■
Signal dropout	■■■■■
Navigation data	■■■■■

Overall:
GOOD FOR INFERENCE

Then dynamically change behavior:

GOOD DATA
→ normal inference

MEDIUM DATA
→ lower-confidence mode

POOR DATA
→ warning + conservative detection

This fits perfectly with your current “graceful degradation” idea.

7. Adaptive confidence threshold

Do not use one fixed confidence threshold for every sonar condition.

For example:

Good quality sonar → threshold 0.45
Medium quality → threshold 0.60
Poor quality → threshold 0.75

The exact values should be learned/tuned from your validation data rather than invented.

So the system becomes:

Sensor condition → adaptive inference policy

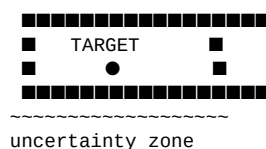
rather than:

Image → YOLO → box

8. Location uncertainty — take it further

Your PPT already mentions location error circles.

Turn that into a proper map feature:



And show:

Estimated position: 21.4721, 72.1842

Estimated uncertainty: ± 8.4 m

Then explain the sources:

- navigation quality
- sonar geometry
- range
- towfish position
- image localization

This is much more defensible than pretending GPS gives exact coordinates.

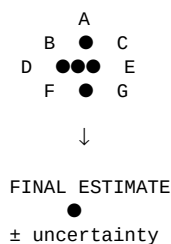
9. Multi-frame geolocation fusion

This is an advanced extension of #3.

If the same object is detected across 10 frames:

Detection 1 → location A
 Detection 2 → location B
 Detection 3 → location C
 ...
 Detection 10 → location J

Fuse them:



So instead of using one noisy image position, your final target location is derived from multiple observations.

That is a very good **research-looking feature**.

10. “Unknown Anomaly” class

Do not force everything into:

- ghost net
- wreck
- pipe
- cylinder
- plane

Add:

UNKNOWN MAN-MADE ANOMALY

This is important because the PS itself is broader than only ghost nets; it is about marine debris and anomalies. ([SIH 2026 Problem Statements](https://sih2026.vuce.in/ps/SIH26057?utm_source=chatgpt.com "SIH26057 · AI-Powered Automated Underwater Marine Debris and Anomaly Detection System using Side-Scan Sonar Imagery | SIH 2026 Problem Statements"))

So:

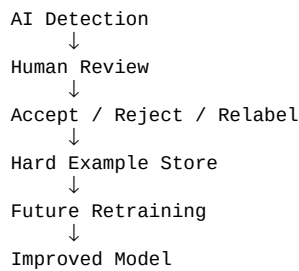
Known
 ■■■ Ghost Net
 ■■■ Wreck
 ■■■ Pipe
 ■■■ Cylinder

Unknown
 ■■■ Possible Anthropogenic Object

Then experts can label the unknown later.

11. Human corrections become future training data

This is a strong AI lifecycle:



This creates:

Continuous Learning Loop

Your application is no longer static.

Every operator decision can produce valuable hard examples.

12. Build a “Hard Negative Bank”

This is a very practical ML trick.

Store difficult false positives:

```
ROCK
RIPPLE
RIDGE
SHADOW
SEDIMENT
SONAR ARTIFACT
VEGETATION
NOISE
```

Then regularly retrain using:

Real object + hard negative

This is more valuable than simply adding thousands of easy images.

Your current project already recognizes the importance of natural seabed hard negatives.

13. Two-stage inference = excellent jugaad

Instead of running the expensive model on everything:

Stage 1 — Fast scan

Low-resolution / lightweight model:

```

Huge sonar file
  ↓
Fast candidate scan
  ↓
Potential regions only

```

Stage 2 — Detailed analysis

```
graph TD; A[Candidate crop] --> B[High-resolution model]; B --> C[Detection + verification];
```

So:

cheap first pass → expensive second pass

This can improve speed and resource use.

...

14. Overlapping tile inference + merge

Huge sonar images are difficult to process as one image.

Do:

A 6x16 grid diagram. The first row is entirely black. The second row contains a black square at column 1, the number 1 at column 2, a black square at column 3, the number 2 at column 4, a black square at column 5, the number 3 at column 6, a black square at column 7, and the number 4 at column 8. The third row is entirely black. The fourth row contains a black square at column 1, the number 4 at column 2, a black square at column 3, the number 5 at column 4, a black square at column 5, the number 6 at column 6, a black square at column 7, and the number 7 at column 8. The fifth row is entirely black. The sixth row is entirely black.

Use overlap:

```
1 + 2 overlap
2 + 3 overlap
...
```

Then merge duplicate detections.

This is a very practical implementation trick.

...

15. Range-aware preprocessing

SSS images change significantly with distance/range.

So don't blindly normalize the entire image.

Split processing based on range:

Near range
↓

Middle range
↓
Far range

Apply different enhancement or model behavior depending on range.

This is much more sonar-aware than generic computer vision preprocessing.

16. Canonicalize port/starboard orientation

This is an underrated practical trick.

SSS surveys may contain different orientations and left/right channels.

Normalize them into a consistent representation before training.

Conceptually:

PORT image → canonical orientation
STARBOARD image → canonical orientation

That can reduce unnecessary variation for the model.

17. Synthetic ghost-net generation — but do it intelligently

You can generate additional training examples by compositing a target with:

- real seabed texture
- realistic intensity
- an associated acoustic-shadow structure
- scale/range variation
- blur/noise/dropout

Concept:

REAL SEABED
+
TARGET STRUCTURE
+
SHADOW
+
SONAR NOISE
↓
SYNTHETIC TRAINING EXAMPLE

But **do not claim synthetic accuracy as real-world accuracy**. A current public SIH26057 implementation itself labels synthetic ghost-net evaluation as synthetic-on-synthetic rather than field performance. ([Hugging Face](https://huggingface.co/datasets/rehan9599/drishti-sss?utm_source=chatgpt.com "rehan9599/drishti-sss · Datasets at Hugging Face"))

That is exactly the kind of distinction you should preserve.

18. Survey coverage / blind-spot map

This is excellent for the GIS side.

Show:

GREEN = surveyed
YELLOW = uncertain coverage
RED = target detected
GREY = not surveyed

Now your system doesn't just answer:

"Where is the debris?"

It also answers:

"Where have we actually looked, and where haven't we?"

That is operationally useful.

19. Repeat-survey change detection

This could become a future/advanced module.

Survey the same area twice:

SURVEY 1
↓
TARGET MAP

SURVEY 2
↓
TARGET MAP

↓

CHANGE ANALYSIS

Then:

NEW TARGET
REMOVED TARGET
PERSISTENT TARGET
LOCATION CHANGED

This turns GhostNet-AI into a **marine monitoring platform**, not only a one-time detector.

20. Recovery Mission Planner

After detection:

```
Detected targets
↓
Priority
↓
Generate recovery route
```

Example map:

```
START
↓
Target #12
↓
Target #24
↓
Target #31
↓
BASE
```

Your AI doesn't perform the recovery itself. It provides a **recommended survey/recovery sequence**.

Label it clearly as an operational planning heuristic, not an autonomous navigation guarantee.

21. “Priority” should be explainable

Don't create a mysterious AI score like:

```
Risk = 93
```

Instead:

```
OPERATIONAL PRIORITY

Target confidence      High
Ghost-net likelihood   High
Persistence            High
Location uncertainty    Low
Survey accessibility   Unknown

Priority:
HIGH

Reason:
High-confidence repeated detection
with low location uncertainty.
```

This is much more defensible.

22. Evidence replay mode

This would be very impressive in a demo.

Click a target:

```
TARGET #24

Frame 102 ■■■■
```

Frame 103 ■■■
Frame 104 ■■■
Frame 105 ■■■
Frame 106 ■■■

↓

Same target

Then show the synchronized map position.

Judge experience:

Click target → watch it across the sonar track → see its map movement → see final consolidated target.

That is a memorable demo.

23. “What changed?” mode

After an operator edits the AI result:

AI:
Ghost Net – 76%

Human:
Rejected

Reason:
Natural seabed ridge

↓
Stored as hard negative

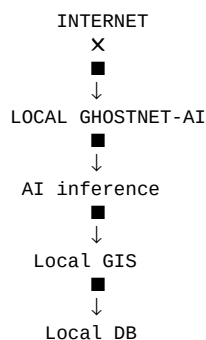
This is excellent for your human-in-the-loop story.

24. Offline / edge mode

Keep this.

Your architecture already plans Docker and modular deployment.

Make the UX demonstrate:



When connectivity returns:

```
LOCAL DATA
↓
SYNC
↓
CENTRAL DATABASE
```

That is far more compelling than simply saying “works offline.”

25. Data provenance / survey evidence chain

Because this is potentially used by researchers/agencies, maintain:

```
SOURCE FILE
↓
SURVEY ID
↓
TIMESTAMP
↓
MODEL VERSION
↓
DETECTION
↓
HUMAN REVIEW
↓
FINAL RECORD
```

For every result:

```
Model: GhostNet-AI v0.8
Inference: 24-09-2026
Source: survey_014.xtf
Reviewer: operator_02
Status: verified
```

That gives your platform a professional scientific workflow.

My “jugaad” toolbox

These are comparatively easy tricks that can make the system significantly better without requiring a giant research lab.

Trick	Why
Overlapping tiles	Reduce missed boundary targets
Two-stage inference	Save compute
Test-time augmentation	Improve robustness
Class-specific thresholds	Different objects behave differently
Hard-negative mining	Reduce repeated false positives
Pseudo-labeling	Exploit useful unlabeled sonar
Spatial/site-based split	Prevent leakage
Multi-frame tracking	Reduce duplicate detections
Range-aware processing	Handle scale/range changes
Left/right canonicalization	Reduce orientation variation

Then visually:

```
RAW SONAR
↓
DETECT
↓
VERIFY
↓
TRUST
↓
LOCALIZE
↓
PRIORITIZE
↓
ACT
```

That gives you a much stronger identity.

One final reality check

I would **not claim** that any one of these features is globally unique unless you have done a literature/product comparison. Recent SSS research already covers areas such as shadow-aware detection, target-height estimation and improved YOLO architectures.

([ScienceDirect](https://www.sciencedirect.com/science/article/pii/S0029801825014179?utm_source=chatgpt.com
"Side-scan sonar mine-like target detection considering acoustic illumination and shadow characteristics - ScienceDirect"))

Your advantage should therefore be **the combination and the working implementation**:

sonar-aware processing + evidence fusion + abstention + multi-frame geolocation + human review + operational GIS workflow.

That combination is what I would make the center of your SIH26057 story—not simply “we use YOLO11-S.”